

Rules for Code

- 1) To count a number in a vector(v) or to count to character('c') in a string(s) use the syntax `count(s.begin(), s.end(), 'c')`.

```
vector<int> ord(n);
iota(ord.begin(), ord.end(), 0);
sort(ord.begin(), ord.end(), [&](int x, int y) -> bool{
    return b[x] < b[y];
});
```

Note

- XOR of all numbers from 1 to n follows a simple pattern:

sql

 Copy code

```
n % 4 == 0 -> result = n
n % 4 == 1 -> result = 1
n % 4 == 2 -> result = n + 1
n % 4 == 3 -> result = 0
```

Note 1: If the number of elements is even, then on performing xor operation twice between them all of them becomes, zero.

Note 2: Suppose we want to find the largest interval between 1 and N which, in which all the elements within the interval divides N, then

Suppose you find a valid interval $[l, r]$. Note that the interval $[l, r]$ contains at least one multiple of x for each $1 \leq x \leq r - l + 1$ (you can find it out by looking at the values in $[l, r]$ modulo x). Then, the interval $[1, r - l + 1]$ is also valid and has the same length.

So, it's enough to check intervals with $l = 1$, i.e., find the smallest x that does not divide n . The answer is $x - 1$.

Complexity: $O(\log(\max n))$

Pow()-> this function gives "double" value and putting that value in the form of int will give precision error of large numbers even if output of Pow() function is integer.

Gray Code Logic:

Recursive logic:

If we know the Gray code for $n-1$,
then Gray code for n is built as:

1. Take all $(n-1)$ -bit codes and prefix them with '0'
2. Take all $(n-1)$ -bit codes in reverse order and prefix them with '1'

That ensures only one bit changes between adjacent entries.

🧠 Example for $n=3$

Gray(2):

```
00, 01, 11, 10
```

[Copy code](#)

Now for $n=3$:

Prefix 0 → 000, 001, 011, 010

Prefix 1 (reverse) → 110, 111, 101, 100



Final sequence:

Formula logic:

🌟 Alternate (Bitwise Formula)

Gray code of integer i can be computed directly as:

```
gray(i) = i ^ (i >> 1)
```

[Copy code](#)

For finding a to the power b :

Instead, we use **Binary Exponentiation (Fast Power)**:

$$a^b = \begin{cases} 1, & b = 0 \\ (a^{b/2})^2, & b \text{ even} \\ a \times (a^{b-1}), & b \text{ odd} \end{cases}$$

```

int result = 1;
while (b > 0) {
    if (b & 1)
        result = (result * a) % MOD;
    a = (a * a) % MOD;
    b >>= 1;
}
return result;

```

♦ Fermat's Little Theorem

If p is prime (and $10^9 + 7$ is prime), then:

$$a^{p-1} \equiv 1 \pmod{p}$$

$$a^E \bmod p = a^{E \bmod (p-1)} \bmod p$$

Sieve of Eratosthenes →

algorithm Sieve of Eratosthenes **is**

input: an integer $n > 1$.

output: all prime numbers from 2 through n .

let A be an array of **Boolean** values, indexed by integers 2 to n , initially all set to **true**.

for $i = 2, 3, 4, \dots$, not exceeding $\sqrt{n}$ **do**

if $A[i]$ **is true**

for $j = i^2, i^2+i, i^2+2i, i^2+3i, \dots$, not exceeding n **do**
 set $A[j] := \text{false}$

return all i such that $A[i]$ **is true**.

```

int main() {
    vector<int>a(1e8,1);
    for(int i = 2; i*i<=1e8; i++){
        if(a[i]==1){
            for(int j = i*i; j<= 1e8 ; j+= i){
                a[j] = 0;
            }
        }
    }
    for(int i = 0; i<a.size(); i++){
        if(a[i]==1){
            cout<<i<<"\n";
        }
    }
}

```

Counting Divisors:

```

void solve(){
    int x;
    cin>>x;
    int divisors = 1;
    for(int i = 2; i*i<=x; i++){
        if(x%i==0){
            int count = 0;
            while(x%i==0){
                count++;
                x = x/i;
            }
            divisors *= (count + 1);
        }
    }
    if(x>1) divisors *= 2;
    cout<<divisors<<"\n";
}

```

(Here we have multiplied with 2 at the end in case x remains greater than 1 at the end).

Finding Fabonacci number in most optimized way:

$$\begin{aligned}
 F(2k) &= F(k) * (2F(k+1) - F(k)) \\
 F(2k+1) &= F(k)^2 + F(k+1)^2
 \end{aligned}$$

↳ square of f(k)

```

7  pair<ll,ll>fabonacci(ll n){
8      if(n==0)return {0,1};
9      auto p = fabonacci(n>>1);
10     ll a = p.first;
11     ll b = p.second;
12     ll c = (a*(2*b%MOD-a + MOD)%MOD)%MOD;
13     ll d = ((a*a%MOD) + (b*b%MOD))%MOD;
14     if(n&1)return {d,(c+d)%MOD};
15     else return {c,d};
16 }

```

Sorting the 2d array (having pair,vector inside)--->

```

sort(v.begin(), v.end(), [](auto &a, auto &b){
    if(a.first!=b.first){
        return a.first<b.first;
    }
    else{
        a.second >b.second;
    }
});

```

To find wheter a particular substring exist in a string or not-

```

if(s.find("2025") != string::npos){
    if(s.find("2026") != string::npos) {
        cout << 0 << nline;
        return;
    }
    cout << 1 << nline;
}

```

→ To find the substring starting from a particular positionⁱ having length l ,
`s.substr(i, l)`

→ In map we cannot update the value of the key. We can only update the value in <key, value> pair.

→ To find the number of set bits in a number — ^{number} `builtin_popcount(x)`;

→ To find the last element of the set use — ^{→ it is $O(1)$ operation.}
`int ele = *st.rbegin()` | to find first element — `int ele = *st.begin()`
or `int ele = *it`

→ To delete an element in a vector use — (using index)
`v.erase(v.begin()+i);`

→ To delete all the occurances of a number — `v.erase(remove(v.begin(), v.end(), num), v.end());`

☆☆ To check whether an element x exists in a vector (arr) or not —
`if (find(arr.begin(), arr.end(), x) == arr.end()) {`
`// element doesn't exist.`
`}`

☆☆ `auto it = lower_bound(arr.begin(), arr.end(), x)` → gives the iterator to the _{first} element which is $\geq x$.

☆☆ to find index use — `int idx = it - arr.begin();`
 If we want to find a _{highest} index of the number which is $\leq x$, then that index is `idx - 1`.

☆☆ `auto it = upper_bound(arr.begin(), arr.end(), x)` → gives the iterator to the first element which is $> x$.

☆☆ If we have an element x in the set, we want to find an element which is before and after this element in the set —

`auto it = st.find(x);`
`auto prev_ele = prev(it);` `previous_element = *prev_ele`
`auto next_ele = next(it);` `next_element = *next_ele`

☆☆☆ To remove the duplicates from the vector, we use —

`v.resize(unique(v.begin(), v.end()) - v.begin())`

1. What happens without `&` ?

cpp

Copy code

```
for (auto it : mpp)
```

What is happening?

- `it` is a **COPY** of each element in the container
- Any change you make affects **only the copy**
- Original container remains unchanged

Example:

cpp

Copy code

```
for (auto it : mpp) {  
    it.second += 10; // modifies copy ❌  
}
```

➡ `mpp` stays the same.

2. What happens with `&` ?

cpp

Copy code

```
for (auto &it : mpp)
```

What is happening?

- `it` is a **REFERENCE** to the actual element inside the container
- No copy is created
- Any allowed modification affects the container directly

Example:

cpp

Copy code

```
for (auto &it : mpp) {  
    it.second += 10; // modifies map value ✅  
}
```

➡ `mpp` is changed.

→ Similarly, we cannot update the value in `set`/`Multiset`, as the values stored in the set is 'key' itself.

Why is & important?

Two reasons:

1. **Efficiency** – avoids copying
2. **Modification** – allows changing original elements

Note-1 In `for(auto it = mpp.begin(); it != mpp.end(); it++)` → 'it' is iterator.
So, to access the elements, we will use 'it → first', 'it → second'

2) In Range based loop –
`for(auto it : mpp)`, here 'it' isn't iterator, it is the copy of the element.
To access the element use `it.first`, `it.second`.

3) When we are doing —

i) `auto it = st.find(x)`, or `auto it = lower_bound(x)`, in all such cases 'it' is iterator, and element will be `*it`.
or `map.find(y)` → key

5 Summary table (VERY IMPORTANT)			
Code	What is <code>it</code>	Iterator or element?	What is <code>*it</code>
<code>auto it = st.lower_bound(x)</code>	<code>set<int>::iterator</code>	iterator	element (const int)
<code>auto it = st.find(x)</code>	<code>set<int>::iterator</code>	iterator	element (const int)
<code>for(auto it : st)</code>	<code>int</code>	element copy	✗ invalid
<code>for(auto &it : st)</code>	<code>const int&</code>	element reference	✗ invalid

→ To find the lcm of more than two numbers —
use $\text{lcm}(a,b) = \frac{a}{\text{gcd}(a,b)} * b$

```
ll lcm_ll(ll a, ll b) {  
    return a / gcd_ll(a, b) * b;  
}  
  
// Compute LCM of all k[i]  
ll z = 1;  
for(int v : k){  
    z = lcm_ll(z, (ll)v);  
}
```

6 Golden rule (memorize this) 🔑

If the RHS is a function like `begin()`, `find()`, `lower_bound()` → `it` is an iterator

If the RHS is a container in a range-based loop → `it` is an element

☆☆
Note. On doing — `auto it = st.find(x)`, Now 'it' is iterator.
So, on doing '`it--`' the iterator will move to the previous element in the set. (if 'it' wasn't equal to `st.begin()`).

☆ To find median of a zero-indexed array → sort the array in increasing order then, $\text{median} = \text{arr}[\frac{n}{2}]$, $n = \text{size of the array}$.

Floating point Precision Error— While using 'double' datatype we need to keep in mind that, when we store —

Ex. `double value = 0.0`, computer store in in the form
`double value = 1e-12`.

Ex2 `double x = 0.1 + 0.2`
`cout << x;` // prints 0.300000000001.

So, On the place where we have boundary conditions like `value > 0`, `value = 0`, or any precision boundaries, we should avoid using double data type.

4] Area of the polygon with vertices $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ is —

$$\frac{1}{2} \left[\begin{vmatrix} x_1 & x_2 \\ y_1 & y_2 \end{vmatrix} + \begin{vmatrix} x_2 & x_3 \\ y_2 & y_3 \end{vmatrix} + \begin{vmatrix} x_3 & x_4 \\ y_3 & y_4 \end{vmatrix} + \dots + \begin{vmatrix} x_n & x_1 \\ y_n & y_2 \end{vmatrix} \right]$$

☆☆
→ If we greater data type than 'long long' then we can use 'long double' (it's range is greater than 10^{30}), also we can use 128-bit data type '`__int128`' (but this datatype will be compiled only on codeforces compiler).

The kth number which is not a perfect square is: $\lfloor \sqrt{k} + 0.5 \rfloor$

** To do prime factorization—
Slow method—

```
for(int i = 2; i<=dup; i++){
    if(dup<=1)break;
    int cnt = 0;
    while(dup>1 && dup%i==0){
        cnt++;
        dup = dup/i;
    }
    if(cnt>0){
        fct.push_back({i,cnt});
    }
}
```

Ex $\Rightarrow 10 = 2^1 \times 5^1$
[{2,1}, {5,1}]

fast method

```
12 for(int i = 2; i*i <= n; i++){
13     if(n % i == 0){
14         int cnt = 0;
15         while(n % i == 0){
16             n /= i;
17             cnt++;
18         }
19         fct.push_back({i, cnt});
20     }
21 }
22 * if(n > 1) fct.push_back({n, 1});
```

In permutations:

- Same $\rightarrow$ reverse $\rightarrow$ rotations $\rightarrow$ arbitrary
Usually the answer appears very early.

Strategy for Questions

1) Whenever there is mentioned that (non-decreasing)/(non-increasing), think also in the way that elements may be equal (Usually in case of subsequences).

2) If $a \oplus b = x$, then $a \oplus x = b$.

When you see:

- maximize OR / AND / XOR
- sum constraint

Immediately think:

"What is the cheapest way to buy bits?"

This single thought unlocks many CF B problems.

3)

4) When we are asked to sort the array with certain operations, always look that, we have to do the initial indices elements as low as possible, and also they should be greater than or equal to the previous elements, we should work in this direction, in such questions.

7 Common patterns to remember (CF GOLD) 🧠

Task	Use
first $\geq x$	lower_bound
first $> x$	upper_bound
last $\leq x$	upper_bound - 1
last $< x$	lower_bound - 1
count of x	upper_bound - lower_bound

5)

6) The number of divisors of a perfect square is always odd.

7) Two the gcd of two consecutive natural numbers is always 1.
Also the two consecutive natural numbers are always co-primes.

8) . If a number is producing same remainder with two numbers ($a \% x = b \% x$) then the divisor of that number will also produce the same remainder ($a \% y = b \% y$, where y is the divisor of x).

9) If we have $2n$ number then it is ways possible to divide them into $n-1$ pairs such that the sum of each pair is divisible by 2.

10) $\gcd(b_1, b_2, \dots, b_m) \leq \gcd(b_1 + b_2, b_3, \dots, b_m)$

11)

One of the possible constructions is the following: let $x = \underbrace{10..00}_{a-c}..0_{c-1}$ and $y = \underbrace{11..10}_{b-c}..0_{c-1}$. Since $10..0$ and $11..1$ are pairwise prime, gcd is 10^{c-1} .

(if x has a digits, y has b digits, and gcd of x, y has c digits.

12)

• $a_i \bmod 2^j$ corresponds to the last j digits in the binary representation of a_i .

13) $a + k * c$ is always divisible by $k+1$ if we take $c = a \% k$.

14) if $s_0 > s_1$

, then $2^{s_0} \geq 2^{s_1} + 2^{s_2}$

15) Even-length palindromes numbers (except 11) are NOT prime.

Higher Rated

1). In prefix sum array, if there is repetition of an element, there must be a contiguous subarray in the original array such that the sum of it's element will be zero.

Introductory Problems

Number Spiral

TASK SUBMIT RESULTS ANALYSIS STATISTICS TESTS QUEUE

Time limit: 1.00 s Memory limit: 512 MB

A number spiral is an infinite grid whose upper-left square has number 1. Here are the first five layers of the spiral:

1	2	9	10	25
4	3	8	11	24
5	6	7	12	23
16	15	14	13	22
17	18	19	20	21

Your task is to find out the number in row y and column x .

Input

The first input line contains an integer t : the number of tests.

After this, there are t lines, each containing integers y and x .

Output

For each test, print the number in row y and column x .

Introductory Problems

...	
Repetitions	✓
Increasing Array	✓
Permutations	✓
Number Spiral	✓
Two Knights	-
Two Sets	-
Bit Strings	-
Trailing Zeros	-
...	

Your submissions

2025-11-05 13:42:09	✓
---------------------	---

```

5 void solve(){
6     ll x,y;
7     cin>>x>>y;
8     ll maxi = max(x,y);
9     ll dia_ele = maxi*maxi-(maxi-1);
10    if(maxi%2==0){
11        if(x<maxi){
12            ll num = dia_ele - (maxi-x);
13            cout<<num<<"\n";
14        }
15        else{
16            ll num = dia_ele + (maxi-y);
17            cout<<num<<"\n";
18        }
19    }
20    else{
21        if(x<maxi){
22            ll num = dia_ele + (maxi-x);
23            cout<<num<<"\n";
24        }
25        else{
26            ll num = dia_ele - (maxi-y);
27            cout<<num<<"\n";
28        }
29    }
30 }

```

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

You have two coin piles containing a and b coins. On each move, you can either remove one coin from the left pile and two coins from the right pile, or two coins from the left pile and one coin from the right pile.

Your task is to efficiently find out if you can empty both the piles.

Input

The first input line has an integer t : the number of tests.

After this, there are t lines, each of which has two integers a and b : the numbers of coins in the piles.

Output

For each test, print "YES" if you can empty the piles and "NO" otherwise.

```

void solve(){
    int a,b;
    cin>>a>>b;
    if(max(a,b)>2*min(a,b) || (a+b)%3 != 0){
        cout<<"NO"<<"\n";
    }

    else{
        cout<<"YES"<<"\n";
    }
}

```

M. Minimum LCM

time limit per test: 2 seconds

memory limit per test: 512 megabytes

You are given an integer n .

Your task is to find two positive (greater than 0) integers a and b such that $a + b = n$ and the least common multiple (LCM) of a and b is the minimum among all possible values of a and b . If there are multiple answers, you can print any of them.

Input

The first line contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 10^9$).

Output

For each test case, print two positive integers a and b — the answer to the problem. If there are multiple answers, you can print any of them.

```

long long a = 1, b = n - 1;

for (long long i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        long long d1 = i;
        long long d2 = n / i;

        if (d1 <= n / 2) {
            a = max(a, d1);
        }
        if (d2 <= n / 2) {
            a = max(a, d2);
        }
    }
}

b = n - a;
cout << a << " " << b << "\n";

```

Restaurant Customers (To find the no. of overlapping intervals) —

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

You are given the arrival and leaving times of n customers in a restaurant.

What was the maximum number of customers in the restaurant at any time?

Input

The first input line has an integer n : the number of customers.

After this, there are n lines that describe the customers. Each line has two integers a and b : the arrival and leaving times of a customer.

You may assume that all arrival and leaving times are distinct.

Output

Print one integer: the maximum number of customers.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a < b \leq 10^9$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4
5 void solve() {
6     int n;
7     cin>>n;
8     vector<pair<int,int>>>v;
9     for(int i = 0; i<n; i++){
10         int a,b;
11         cin>>a>>b;
12         v.push_back({a,1});
13         v.push_back({b,-1});
14     }
15     sort(v.begin(), v.end(),[](auto &x, auto &y){
16         if(x.first==y.first)return x.second>y.second;
17         else return x.first<y.first;
18     });
19     int count = 0;
20     int max_count = 0;
21     for(auto &events:v){
22         count += events.second;
23         max_count = max(max_count,count);
24     }
25     cout<<max_count<<"\n";
26 }
27
28 int main() {
29     // int t;
30     // cin >> t;
31     // while (t--) solve();
32     solve();
33     return 0;
34 }
```

Movie Festival (To find the number of non-overlapping intervals).

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [STATISTICS](#) | [TESTS](#)

Time limit: 1.00 s **Memory limit:** 512 MB

In a movie festival n movies will be shown. You know the starting and ending time of each movie. What is the maximum number of movies you can watch entirely?

Input

The first input line has an integer n : the number of movies.

After this, there are n lines that describe the movies. Each line has two integers a and b : the starting and ending times of a movie.

Output

Print one integer: the maximum number of movies.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a < b \leq 10^9$

Example

Input:

```
3
3 5
4 9
5 8
```

Output:

```
2
```

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4
5 void solve() {
6     int n;
7     cin>>n;
8     vector<pair<int,int>>>v;
9     for(int i = 0; i<n; i++){
10         int a,b;
11         cin>>a>>b;
12         v.push_back({a,b});
13     }
14     sort(v.begin(), v.end(), [](auto &x, auto &y){
15         if(x.second==y.second)return x.first<y.first;
16         else return x.second<y.second;
17     });
18     int count = 1;
19     int max_count = 1;
20     int last = v[0].second;
21     for(int i = 1; i<n; i++){
22         if(v[i].first>=last){
23             count++;
24             max_count = max(max_count, count);
25             last = v[i].second;
26         } } skipping when v[i].first < last
27     cout<<max_count<<"\n";
28 }
29
30
31 int main() {
32     // int t;
33     // cin >> t;
34     // while (t--) solve();
35     solve();
36     return 0;
37 }

```

Maximum Subarray Sum

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

Given an array of n integers, your task is to find the maximum sum of values in a contiguous, nonempty subarray.

Input

The first input line has an integer n : the size of the array.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Output

Print one integer: the maximum subarray sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq x_i \leq 10^9$

Example

Input:

8
-1 3 -2 5 3 -5 2 2

Output:

9

```

5 void solve() {
6     int n;
7     cin>>n;
8     vector<int>x(n);
9     for(int i = 0; i<n; i++){
10         cin>>x[i];
11     }
12     int k=-1;
13     for(int i = 0; i<n; i++){
14         if(x[i]>0){
15             k = i;
16             break;
17         }
18     }
19     int low = k;
20     int high = k;
21     ll sum = 0;
22     ll max_sum = 0;
23     while(high<n){
24         sum += x[high];
25         max_sum = max(max_sum,sum);
26         if(sum<0){
27             while(low<=high && sum<=0){
28                 sum -= x[low];
29                 low++;
30                 max_sum = max(max_sum,sum);
31             }
32         }
33         high++;
34     }
35     if(k==-1)max_sum = *max_element(x.begin(), x.end());
36     cout<<max_sum<<"\n";
37 }
38 }

```

Missing Coin Sum

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

You have n coins with positive integer values. What is the smallest sum you cannot create using a subset of the coins?

Input

The first line has an integer n : the number of coins.

The second line has n integers $x_1, x_2, \dots, x_n$: the value of each coin.

Output

Print one integer: the smallest coin sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Example

Input:

```
5
2 9 1 2 7
```

Output:

```
6
```

```
5 void solve() {
6     int n;
7     cin>>n;
8     vector<int>v(n);
9     for(int i = 0; i<n; i++){
10         cin>>v[i];
11     }
12     sort(v.begin(), v.end());
13     ll missing = 1;
14     for(auto & x: v){
15         if(missing<x){
16             break;
17         }
18         else{
19             missing += x;
20         }
21     }
22     cout<<missing<<"\n";
23
24
25 }
26
27 int main() {
28     // int t;
29     // cin >> t;
30     // while (t--) solve();
31     solve();
32     return 0;
33 }
```

Collecting Numbers

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an array that contains each number between $1 \dots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order.

On each round, you go through the array from left to right and collect as many numbers as possible. What will be the total number of rounds?

Input

The first line has an integer n : the array size.

The next line has n integers $x_1, x_2, \dots, x_n$: the numbers in the array.

Output

Print one integer: the number of rounds.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$

Example

Input:

```
5
4 2 1 5 3
```

Output:

```
3
```

```
5 void solve() {
6     int n;
7     cin >> n;
8     vector<int> v(n), pos(n+1);
9     for(int i = 0; i < n; i++){
10         cin >> v[i];
11         pos[v[i]] = i;
12     }
13     pos[0] = 0;
14     int count = 1;
15     vector<int> vis(n+1, -1);
16     for(int i = 0; i < n; i++){
17         if(pos[v[i]] < pos[v[i]-1]) count++;
18     }
19     cout << count << "\n";
20 }
21
```

Collecting Numbers II

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an array that contains each number between $1 \dots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order.

On each round, you go through the array from left to right and collect as many numbers as possible.

Given m operations that swap two numbers in the array, your task is to report the number of rounds after each operation.

Input

The first line has two integers n and m : the array size and the number of operations.

The next line has n integers $x_1, x_2, \dots, x_n$: the numbers in the array.

Finally, there are m lines that describe the operations. Each line has two integers a and b : the numbers at positions a and b are swapped.

Output

Print m integers: the number of rounds after each swap.

Constraints

```

5 void solve() {
6     int n, m;
7     cin >> n >> m;
8     vector<int> a(n + 1), pos(n + 1);
9     for (int i = 1; i <= n; i++) {
10         cin >> a[i];
11         pos[a[i]] = i;
12     }
13
14     int rounds = 1;
15     for (int i = 2; i <= n; i++) {
16         if (pos[i] < pos[i - 1])
17             rounds++;
18     }
19
20     while (m--) {
21         int i, j;
22         cin >> i >> j;
23
24         int x = a[i];
25         int y = a[j];
26         set<int> affected = {x, x - 1, x + 1, y, y - 1, y + 1};
27         for (int v : affected) {
28             if (v >= 2 && v <= n) {
29                 if (pos[v] < pos[v - 1])
30                     rounds--;
31             }
32         }
33         swap(a[i], a[j]);
34         pos[x] = j;
35         pos[y] = i;
36         for (int v : affected) {
37             if (v >= 2 && v <= n) {
38                 if (pos[v] < pos[v - 1])
39                     rounds++;
40             }
41         }
42         cout << rounds << "\n";
43     }

```

Dynamic Programming

